

Patan Multiple Campus
Tribhuvan University, Nepal

Master of Computer Application (MCA)

Dissertation Proposal on

Comparative Analysis of Transformers and Linear Models for Electricity Consumption Forecasting

Submitted By:
Anupam Gautam
Roll No: 02/80

Submitted in partial fulfillment of the requirement for the degree of Master in Computer Application (MCA)

Supervisor
Prof. Roshan Tandukar

Abstract

Time series forecasting is a fundamental component of effective energy management, as accurate electricity consumption predictions are essential for resource planning, demand management, and informed decision-making. While deep learning architectures have traditionally been employed to model temporal dependencies, recent advances have led to the increasing adoption of Transformer-based models for long-term time series forecasting. However, the effectiveness of these complex architectures for long-term electricity consumption forecasting remains an active area of academic investigation, as recent studies have demonstrated that simpler linear models can achieve competitive or superior forecasting performance while requiring substantially lower computational resources.

This research conducts a comparative analysis of Transformer-based and linear models for long-term multivariate electricity consumption forecasting. It evaluates DLinear, NLinear, PatchTST, and iTransformer using electricity consumption time-series data, examining the effects of forecasting horizon, lookback window length, and data characteristics. Model performance is assessed using MAE, MSE, and computational efficiency metrics. The study aims to identify the conditions under which the additional complexity of Transformer models provides meaningful advantages over simpler and more efficient linear approaches.

Keyword: *Long-Term Time Series Forecasting (LTSF), Multivariate Electricity Load Forecasting, Transformer-Based Models, Linear Forecasting Models, DLinear, NLinear, PatchTST, iTransformer, Temporal Dependency Modelling*

List of Diagrams

Fig 5.2.1: Working flow state of Decomposition Linear…… 8

Fig 5.2.2: Working flow state of Normalized Linear……. 9

Fig 5.2.3: Working flow state of Patch Time Series Transformer…….……... 10

Fig 5.2.4: Working flow state of Inverted Transformer…….……... 11

Tables of Contents

Introduction 1

Problem Statement 2

Objectives 3

Review of Literature 4

Research Methodology 5

Datasets 5

- Attribute Selection and Generalisability 6
- Data Preprocessing and Experimental Preparation 7

Forecasting Models 7

- Global Mathematical Notation 7
- DLinear (Decomposition Linear) 8
- NLinear (Normalized Linear) 9
- PatchTST (Patch Time Series Transformer) 10
- iTransformer (Inverted Transformer) 11
- Experimental Design 12
- Performance Evaluation 13
- Statistical Analysis 14
- Outcome 14

References 15

Introduction

Time series forecasting is a critical task across various domains, including finance, energy management, and weather prediction. Historically, statistical methods like ARIMA and state-space models dominated the field. With the onset of deep learning, Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks became popular due to their ability to capture sequential dependencies [1]. These approaches significantly improved forecasting accuracy by learning complex nonlinear relationships from historical observations without relying heavily on handcrafted features or strict statistical assumptions. However, despite their effectiveness, RNN- and LSTM-based models often encounter challenges such as vanishing gradients, limited parallelization during training, and difficulty in modeling very long-term dependencies. These limitations have motivated researchers to investigate alternative neural architectures capable of handling increasingly large and complex temporal datasets more efficiently. Recently, the success of Transformer architecture in Natural Language Processing (NLP) has spurred a surge of Transformer-based models for long-term time series forecasting (LTSF). As the volume and complexity of temporal data continue to grow, developing forecasting models that can effectively capture underlying patterns has become an increasingly important research area. The increasing adoption of Internet of Things (IoT) devices, smart sensors, and large-scale monitoring systems has further accelerated the generation of multivariate time series data, creating new challenges related to scalability, computational efficiency, and feature extraction. Consequently, modern forecasting models are expected not only to produce accurate predictions but also to generalize well across different datasets and application domains while maintaining reasonable computational costs. These models, such as Informer, Autoformer, and FEDformer, leverage self-attention mechanisms to capture long-range dependencies [2].

Problem Statement

Despite the architectural sophistication of Transformer models, their effectiveness in time series forecasting remains a subject of considerable debate. A seminal 2023 study, *"Are Transformers Effective for Time Series Forecasting?"*, demonstrated that a simple family of linear models, collectively termed LTSF-Linear (e.g., DLinear), consistently outperformed several state-of-the-art Transformer-based forecasting models across multiple benchmark datasets [3]. The study argued that self-attention is inherently permutation-invariant, making it susceptible to losing temporal ordering information, which is a fundamental property of time series data.

Although techniques such as positional encodings, series decomposition, and patch-based representations have been introduced to address this limitation, they also increase model complexity and computational cost without consistently delivering proportional improvements in forecasting accuracy. Furthermore, existing studies report conflicting findings regarding the conditions under which Transformer architectures outperform simpler linear models, leaving uncertainty about their practical applicability. Consequently, there is a clear need for a systematic evaluation that identifies the specific data characteristics, forecasting horizons, and dataset complexities that justify the additional computational overhead of Transformer-based models compared with efficient linear alternatives. This dissertation addresses this research gap by empirically comparing both model families across diverse time series forecasting scenarios to determine when the use of Transformers is genuinely advantageous.

Objectives

The primary objective of this research is to compare the performance of Transformer-based and linear models for time series forecasting. Specifically, the study aims

- To compare the forecasting performance of DLinear and Patch-based Transformer models on stationary and non-stationary time series data.
- To evaluate the effect of lookback window length and forecasting horizon on the accuracy and scalability of both model families.

Review of Literature

The literature on Transformer-based models for time series forecasting can be broadly categorized into architectural innovations and critical evaluations of their effectiveness. Several studies have focused on improving the efficiency and forecasting capability of Transformer architectures. For instance, Informer [5] introduced the ProbSparse self-attention mechanism to reduce the computational complexity associated with the standard self-attention operation, making it more suitable for long-sequence forecasting tasks. Similarly, Autoformer [2] enhanced forecasting performance by incorporating series decomposition and an Auto-Correlation mechanism, enabling the model to capture long-term dependencies more effectively. More recently, PatchTST [6] proposed a patch-based input representation that preserves local semantic information while reducing the redundancy inherent in point-wise attention mechanisms.

Despite these architectural advancements, the effectiveness of Transformer-based models has been questioned in the context of long-term time series forecasting (LTSF). Zeng et al. [3] argued that the temporal relationships present in many LTSF datasets are sufficiently simple to be modeled effectively by a single linear layer. Their findings suggested that complex multi-head attention mechanisms may unnecessarily increase model complexity, leading to overfitting and a loss of temporal order information, thereby allowing simple linear models to outperform several state-of-the-art Transformer architectures.

In response to these criticisms, subsequent research has challenged the conclusion that Transformers are inherently unsuitable for time series forecasting. The iTransformer model [4] proposed that the limitations of earlier Transformer models were primarily due to their embedding strategy, which fused multiple variables rather than treating individual time points as tokens. By inverting the input dimensions, iTransformer enables the attention mechanism to capture multivariate correlations more effectively while preserving temporal information. This approach demonstrated that, when appropriately designed, Transformer architectures can overcome the shortcomings identified in earlier studies and achieve competitive performance compared with linear models.

Research Methodology

This research will adopt a comparative empirical methodology to investigate the forecasting performance of Transformer-based architecture and linear models under varying time series characteristics. To achieve this objective, experiments will be conducted using both real-world benchmark datasets and synthetically generated datasets, enabling evaluation under practical and controlled environments.

Datasets

The study will use the publicly available Smart Grid Load Forecasting Dataset (2020–2024), available through Kaggle. The dataset contains five years of hourly electricity demand observations simulated for a smart-grid environment representing Nigerian urban power systems, with a nominal base-load capacity of approximately 120 MW. The dataset is designed to represent realistic electricity consumption behaviour and is therefore suitable for evaluating time-series forecasting models under different temporal, environmental, and operational conditions.

Dataset source: Kaggle – Hourly Load Consumption Data

The uploaded dataset contains 43,848 hourly observations and 14 columns. Of these, one column represents the timestamp, one represents the forecasting target (load_kw), and the remaining 12 columns provide potential explanatory attributes describing temporal, environmental, seasonal, and operational conditions. The attributes include hour of day, day of week, weekend status, month, seasonal indicators (dry, rainy, and harmattan seasons), temperature, rainfall-day indicator, national and religious holiday indicators, and lockdown status. The target variable is electricity load measured in kilowatts.

Table 6.1. Complementary modelling assumptions for proposed experiments

Category	Attribute(s)	Role
Temporal	Hour, weekday, month	Capture intra-day, weekly, and annual patterns
Calendar	Weekend, national holiday, religious holiday	Represent changes in consumption associated with calendar events

Seasonal	Dry, rainy, harmattan	Represent seasonal changes in demand
Environmental	Temperature, rain-day indicator	Capture weather-related effects on electricity consumption
Operational	Lockdown indicator	Represent abnormal changes in consumption behaviour
Target	Load (load_kw)	Electricity demand to be forecast

Attribute Selection and Generalisability

Although the dataset provides 12 potential explanatory attributes in addition to the timestamp and target variable, the study will not indiscriminately use every available attribute as an independent model input. A controlled feature-selection strategy will be applied to retain a limited set of theoretically relevant and non-redundant attributes. This is important because the objective of the study is not simply to maximise forecasting accuracy on one dataset, but to investigate under what data characteristics Transformer architectures provide advantages over simpler linear models.

This restriction is intended to improve the generalisability and interpretability of the experiments. Including a large number of potentially redundant or weakly informative attributes can increase the dimensionality of the input space and may allow a model to learn dataset-specific patterns rather than general forecasting relationships. This is particularly relevant for high-capacity Transformer models, which have substantially more parameters and representational capacity than linear forecasting models. Therefore, limiting the number of input attributes helps reduce unnecessary model complexity and provides a more controlled environment for comparing DLinear, NLinear, PatchTST, and iTransformer.

This controlled feature-selection approach is particularly important for the dissertation because it separates architectural complexity from feature complexity. If different models were provided with substantially different information, an observed performance difference could not be confidently attributed to the model architecture itself. By providing all four models with the same selected information, the experiments can more fairly assess whether the additional representational capacity of Transformer architectures provides measurable benefits over simpler linear approaches.

Data Preprocessing and Experimental Preparation

The raw dataset will first be ordered chronologically using the timestamp variable and checked for missing observations, duplicated timestamps, inconsistent values, and anomalous records. Temporal attributes will be converted into suitable numerical or categorical representations where necessary. Continuous variables such as temperature and electricity load will be appropriately scaled using statistics derived from the training data only, thereby preventing information from the validation or test periods from leaking into the training process.

The dataset will subsequently be used to conduct controlled experiments examining the effects of forecasting horizon, lookback-window length, stationarity, distributional shifts, noise, and outliers. These experiments will allow the study to move beyond a simple comparison of average forecasting accuracy and investigate the central research question: under what characteristics of time-series data do Transformer-based models justify their additional architectural complexity compared with simpler linear models?

Forecasting Models

The study will compare four representative forecasting architectures: DLinear, NLinear, PatchTST, and iTransformer. These models are selected to provide a structured comparison between computationally efficient linear forecasting approaches and Transformer-based architectures for long-term multivariate electricity consumption forecasting.

Global Mathematical Notation

For consistency across all models, we define the base forecasting formulation as follows:

- $\mathrm{X} \in \mathbb{R}^{L \times C}$: The historical multivariate input time series.
- L: The look-back window length (historical sequence length).
- C: The number of variables (channels) in the multivariate dataset, such as distinct electricity consumption nodes.
- T: The forecasting horizon (future sequence length).
- $\hat{\mathrm{Y}} \in \mathbb{R}^{T \times C}$: The predicted future values.

- All linear and Transformer models in this study operate under a Channel-Independent (CI) assumption where each variable $c \in \{1,2,\dots,C\}$ is processed independently, unless explicitly stated otherwise (as in the iTransformer).

DLinear (Decomposition Linear)

DLinear decomposes the input time series into trend and seasonal (remainder) components using a moving-average filter before applying independent linear projections.

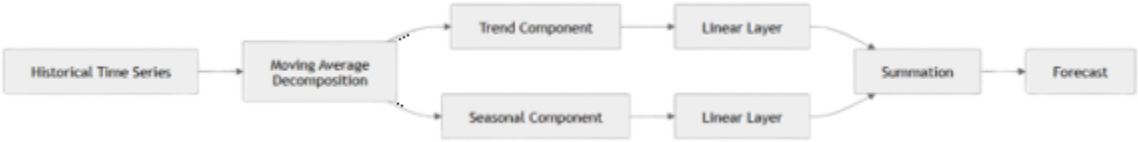


Fig 5.2.1: Working flow state of Decomposition Linear

Mathematical Formulation:

The model first applies a moving average kernel to extract the trend component X_{trend} for each channel c. The operation for a specific time step t is defined as:

$$\mathrm{X}_{trend}^{(c)}(t) = \frac{1}{2k+1} \sum_{j=-k}^{k} \mathrm{X}^{(c)}(t+j)$$

$$\mathrm{X}_{seasonal}^{(c)} = \mathrm{X}^{(c)} - \mathrm{X}_{trend}^{(c)}$$

Each component is then independently projected to the forecasting horizon T via a single-layer linear network:

$$\hat{\mathrm{Y}}_{trend}^{(c)} = \mathrm{W}_t \mathrm{X}_{trend}^{(c)} + \mathrm{b}_t$$

$$\hat{\mathrm{Y}}_{seasonal}^{(c)} = \mathrm{W}_s \mathrm{X}_{seasonal}^{(c)} + \mathrm{b}_s$$

The final prediction is the element-wise addition of the two projected components:

$$\hat{\mathrm{Y}}^{(c)} = \hat{\mathrm{Y}}_{trend}^{(c)} + \hat{\mathrm{Y}}_{seasonal}^{(c)}$$

Where:

- $\mathrm{X}^{(c)} \in \mathbb{R}^{L}$: The historical sequence for a single variable c.
- k: The half-window size of the moving average filter.
- $\mathrm{X}_{trend}^{(c)}, \mathrm{X}_{seasonal}^{(c)} \in \mathbb{R}^{L}$: The extracted trend and seasonal sub-sequences.
- $\mathrm{W}_t, \mathrm{W}_s \in \mathbb{R}^{T \times L}$: The learned weight matrices for the trend and seasonal components, mapping historical length L to future horizon T.
- $\mathrm{b}_t, \mathrm{b}_s \in \mathbb{R}^{T}$: The learned bias vectors.

- $\hat{Y}_{trend}^{(c)}, \hat{Y}_{seasonal}^{(c)}, \hat{Y}^{(c)} \in \mathbb{R}^{T}$: The forecasted trend, seasonal, and final sequences.

NLinear (Normalized Linear)

NLinear is a highly efficient single-layer linear model designed specifically to counter distributional shifts (such as sudden baseline changes in electricity consumption) by performing a localized sequence normalization prior to the linear projection.

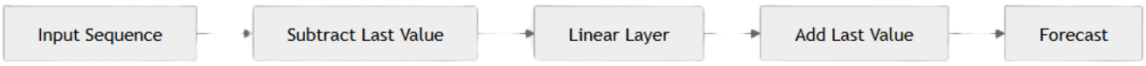


Fig 5.2.2: Working flow state of Normalized Linear

Mathematical Formulation:

The model anchors the sequence by isolating the final observed value of the look-back window and subtracting it across the entire historical sequence:

$$X'^{(c)} = X^{(c)} - \left(x_{last}^{(c)} \cdot 1_L\right)$$

The normalized sequence is then projected across the prediction horizon:

$$\hat{Y}'^{(c)} = WX'^{(c)} + b$$

Finally, the model applies a denormalization step by adding the anchoring value back to the projected sequence:

$$\hat{Y}^{(c)} = \hat{Y}'^{(c)} + \left(x_{last}^{(c)} \cdot 1_T\right)$$

Where:

- $X^{(c)} \in \mathbb{R}^{L}$: The historical sequence for variable c.
- $x_{last}^{(c)} \in \mathbb{R}$: The scalar value at the final time step of the input sequence (i.e., $X_L^{(c)}$).
- $1_L \in \mathbb{R}^{L}$ and $1_T \in \mathbb{R}^{T}$: Column vectors of ones used for broadcasting the scalar subtraction and addition across the time dimensions.
- $X'^{(c)} \in \mathbb{R}^{L}$: The mean-shifted (normalized) input sequence.
- $W \in \mathbb{R}^{T \times L}$: The learned linear projection weight matrix.
- $b \in \mathbb{R}^{T}$: The learned bias vector.
- $\hat{Y}'^{(c)} \in \mathbb{R}^{T}$: The normalized prediction sequence.

PatchTST (Patch Time Series Transformer)

PatchTST bridges the gap between natural language processing and time series by dividing 1D temporal sequences into fixed-length patches. This tokenization strategy preserves local semantic temporal information while drastically reducing the computational complexity of the attention mechanism.



Fig 5.2.3: Working flow state of Patch Time Series Transformer

Mathematical Formulation:

The historical sequence $\mathrm{X}^{(c)} \in \mathbb{R}^{L}$ is segmented into overlapping or non-overlapping patches:

$$\mathrm{X}_{p}^{(c)} \in \mathbb{R}^{N_p \times P}$$

Where the number of patches is calculated as $N_p = \left\lfloor \frac{L-P}{S} \right\rfloor + 1$.

Each patch is mapped to a high-dimensional latent space and augmented with a learnable positional encoding:

$$\mathrm{Z}_0 = \mathrm{X}_{p}^{(c)} \mathrm{W}_{emb} + \mathrm{U}_{pos}$$

The tokens are passed through M standard Multi-Head Self-Attention (MSA) encoder layers:

$$\mathrm{Z}_l' = \mathrm{MSA}\big(\mathrm{LayerNorm}(\mathrm{Z}_{l-1})\big) + \mathrm{Z}_{l-1}$$

$$\mathrm{Z}_l = \mathrm{FFN}\big(\mathrm{LayerNorm}(\mathrm{Z}_l')\big) + \mathrm{Z}_l'$$

Where the attention mechanism for matrices Query (Q), Key (K), and Value (V) is defined as:

$$\mathrm{Attention}(\mathrm{Q}, \mathrm{K}, \mathrm{V}) = \mathrm{softmax}\left(\frac{\mathrm{QK}^T}{\sqrt{d_{model}}}\right)\mathrm{V}$$

Finally, the output is flattened and passed through a multi-layer perceptron (MLP) head to generate the forecast:

$$\hat{\mathrm{Y}}^{(c)} = \mathrm{MLP}\big(\mathrm{Flatten}(\mathrm{Z}_M)\big)$$

Where:

- P: The temporal length of a single patch.
- S: The stride (step size) between consecutive patches.

- N_p: The total number of resulting patch tokens.
- $\mathrm{W}_{emb} \in \mathbb{R}^{P \times d_{model}}$: The linear patch embedding matrix.
- $\mathrm{U}_{pos} \in \mathbb{R}^{N_p \times d_{model}}$: The learnable positional encoding matrix.
- d_{model}: The hidden dimension size of the Transformer architecture.
- $\mathrm{Z}_l \in \mathbb{R}^{N_p \times d_{model}}$: The latent representation at encoder layer $l \in \{1, \ldots, M\}$.
- $\mathrm{Q}, \mathrm{K}, \mathrm{V} \in \mathbb{R}^{N_p \times d_{model}}$: The attention projection matrices.

iTransformer (Inverted Transformer)

Unlike conventional Transformers that treat individual time steps as tokens, the iTransformer inherently captures cross-variate dependencies by inverting the tokenization axis. Entire time series variables (e.g., individual electricity nodes) are embedded as independent tokens, allowing the attention mechanism to map correlations between variables rather than between time steps.



Fig 5.2.4: Working flow state of Inverted Transformer

Mathematical Formulation:

The model transposes the standard input space, treating the entire historical sequence of a single channel c as a single token vector. This sequence is embedded into the latent dimension:

$$\mathrm{h}_c = \mathrm{X}_{:,c}\mathrm{W}_{emb}$$

Resulting in an embedding matrix for all variables:

$$\mathrm{H}_0 = [\mathrm{h}_1, \mathrm{h}_2, \ldots, \mathrm{h}_C]^T + \mathrm{V}_{pos}$$

Self-attention is then applied *across the variables* rather than across time:

$$\mathrm{H}_l{}' = \mathrm{MSA}\big(\mathrm{LayerNorm}(\mathrm{H}_{l-1})\big) + \mathrm{H}_{l-1}$$

$$\mathrm{H}_l = \mathrm{FFN}\big(\mathrm{LayerNorm}(\mathrm{H}_l{}')\big) + \mathrm{H}_l{}'$$

The final prediction is generated by projecting the updated variable tokens directly to the future horizon:

$$\hat{\mathrm{Y}} = \mathrm{H}_M\mathrm{W}_{proj}$$

Where:

- $\mathrm{X}_{:,c} \in \mathbb{R}^L$: The full historical sequence for variable c, extracted as a column from X.

- $W_{emb} \in \mathbb{R}^{L \times d_{model}}$: The embedding weights mapping the temporal length to the Transformer's hidden dimension.
- $h_c \in \mathbb{R}^{d_{model}}$: The embedded token representation of the c-th variable.
- $H_0 \in \mathbb{R}^{C \times d_{model}}$: The combined initial token matrix for all C variables.
- $V_{pos} \in \mathbb{R}^{C \times d_{model}}$: The variate positional embedding matrix.
- M: The number of Transformer encoder layers.
- $H_M \in \mathbb{R}^{C \times d_{model}}$: The final encoded representation after M layers.
- $W_{proj} \in \mathbb{R}^{d_{model} \times T}$: The projection head mapping the hidden dimension to the forecast horizon.

Experimental Design

A series of controlled experiments will be conducted to evaluate the performance of DLinear, NLinear, PatchTST, and iTransformer for long-term electricity consumption forecasting. All models will be trained and evaluated using the same data splits, preprocessing pipeline, and evaluation metrics to ensure a fair comparison.

Experiment 1: Effect of Forecasting Horizon

To investigate how increasing the prediction horizon affects the forecasting accuracy of linear and Transformer-based models.

Experimental Setup

- Forecasting horizons: 96, 192, 336, and 720 time steps.
- All models will be trained using the same lookback window and hyperparameter configuration.
- The forecasting horizon will be the only variable changed during this experiment.

Evaluation

- Compare forecasting performance using MAE and MSE.
- Record computational performance, including training time and inference time.

Expected Analysis

- Determine whether Transformer models provide greater benefits for longer forecasting horizons where long-range temporal dependencies become more significant.

Experiment 2: Effect of Lookback Window Length

To evaluate how the amount of historical information influences forecasting performance.

Experimental Setup

- Train each model using multiple lookback window lengths (e.g., 96, 192, 336, and 720 historical observations).
- Maintain identical forecasting horizon, preprocessing, and hyperparameters across all experiments.

Evaluation

- Compare MAE and MSE for each lookback window.
- Analyse changes in computational cost as the input sequence length increases.

Expected Analysis

- Identify the minimum historical context required by each model.
- Assess whether Transformer architectures benefit more from longer input sequences than linear models.

Experiment 3: Effect of Distributional Shift

To assess model robustness under different statistical characteristics of the time series.

Experimental Setup

- Evaluate models using:
    - Original electricity consumption data.
    - Differenced and normalized data.
- Keep all training configurations identical across both datasets.

Evaluation

- Compare forecasting accuracy using MAE and MSE.
- Perform statistical significance testing using the Diebold–Mariano (DM) test where appropriate.

Expected Analysis

- Determine whether improved stationarity benefits linear models more than Transformer-based models.
- Assess the robustness of each model when the underlying data distribution changes.

Performance Evaluation

Forecasting accuracy will be evaluated using standard regression metrics, including Mean Absolute Error (MAE) and Mean Squared Error (MSE). In addition, Dynamic Time Warping (DTW) will be employed to measure the similarity between predicted and actual

time series by accounting for temporal alignment, providing complementary insight beyond point-wise error metrics.

To assess computational efficiency, training time, inference time, model parameter count, and floating-point operations (FLOPs) will also be recorded. These measures enable evaluation of the trade-off between forecasting accuracy and computational cost, which is central to determining whether the additional complexity of Transformer architectures is practically justified.

Statistical Analysis

Results will be validated using the Diebold-Mariano test to ensure performance differences are not due to chance [7]. This statistical test compares the predictive accuracy of competing forecasting models and determines whether differences in forecasting errors are statistically significant rather than resulting from random variation. The inclusion of statistical significance testing strengthens the validity and reliability of the experimental findings.

Outcome

The expected outcome of this dissertation is a comprehensive framework or "decision matrix" for practitioners. This framework will specify the conditions (e.g., high multivariate correlation, long-range non-linear dependencies) where the deployment of a Transformer is justified. Conversely, it will identify scenarios where linear models provide a more robust and efficient solution. The research will contribute to the theoretical understanding of temporal representation learning and provide practical guidelines for model selection in industrial time series applications.

References

[1] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[2] H. Wu et al., "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in Advances in Neural Information Processing Systems (NeurIPS), 2021.

[3] A. Zeng et al., "Are transformers effective for time series forecasting?" in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, no. 9, pp. 11121–11128, 2023.

[4] Y. Liu et al., "iTransformer: Inverted transformers are effective for time series forecasting," in International Conference on Learning Representations (ICLR), 2024.

[5] H. Zhou et al., "Informer: Beyond efficient transformer for long-term time-series forecasting," in Proceedings of the AAAI Conference on Artificial Intelligence, 2021.

[6] Y. Nie et al., "A time series is worth 64 words: Long-term forecasting with transformers," in International Conference on Learning Representations (ICLR), 2023.

[7] F. X. Diebold and R. S. Mariano, "Comparing predictive accuracy," Journal of Business & Economic Statistics, vol. 13, no. 3, pp. 253–263, 1995.